Open in app ↗ Sign up Sign in

Medium Search

Oop Concepts + Cpp + Software Engineering + Programming +

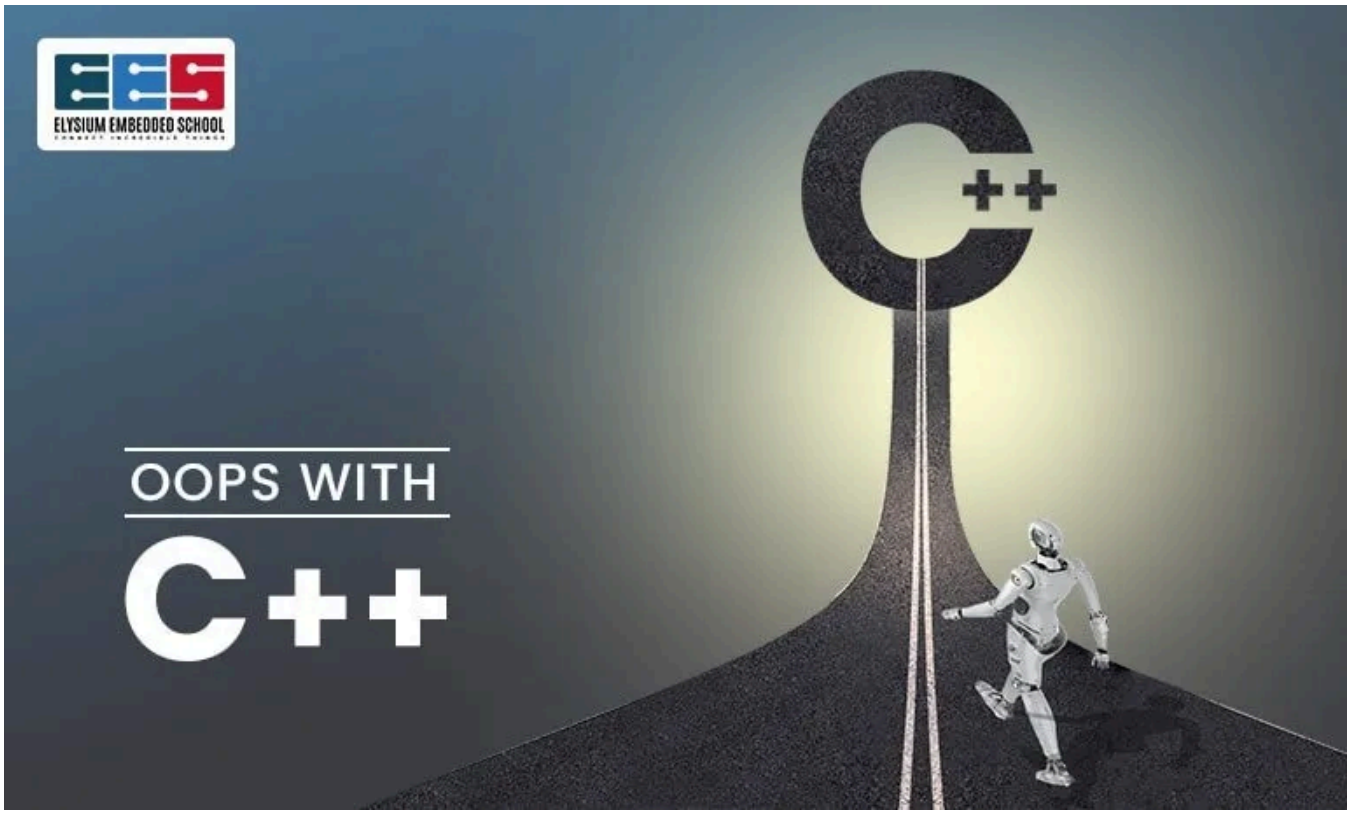
“Mastering Object-Oriented Programming (OOP) in C++: A Comprehensive Guide”

10 min read · Dec 24, 2023



Ankit Singh Follow

Listen Share



Before we start learning about OOP (Object-Oriented Programming), let’s figure out why it’s important. We’ll explore what we used before OOP and what concepts were in use back then.

- Before, we used procedural programming.

Procedural Programming:



- Procedural programming is when a program uses procedures or functions that work with data.
- Procedural programming is a list of instructions on a single block, basically program is divided into tasks (group of functions).
- It uses High-level Languages like C etc.
- It involves using **global data**, which means Data is often shared globally among functions.

Example:



Limitations of Procedural Programming:

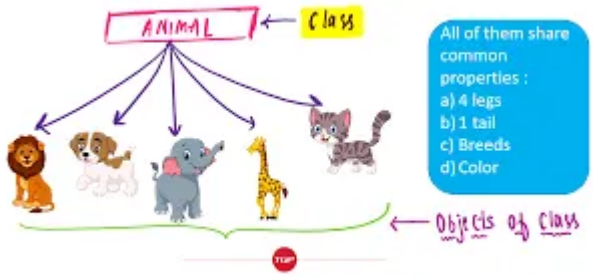
1. **Lack of Data Security:** Global data can be accessed and modified by any function, potentially leading to unintended changes.
2. **Maintenance Complexity:** As the program grows, managing global data and functions can become challenging.
3. **Limited Reusability:** Functions may need to be re-implemented or rewritten for different programs as they are tightly coupled with the data.

> *As software gets bigger and more complex, procedural programming can struggle to handle it well. That's why other ways of programming like OOP and functional programming became popular — they help deal with these problems.*



What is object-oriented programming?

- Object-oriented programming is a technique/methodology/paradigm to design a program using class and objects.
- In this, the main focus was on data, and blinding the data to the function using it.
- Each object has its data and function(method) and the data of those objects is only accessible the their function.



what is class?

- Class is a fundamental unit of OOP and it is a blueprint of objects.
- It is a user-defined datatype, which defines its properties and its function (method).

Example:



what is an object?

- An object is an entity that has a state, property, and behavior.
- The object is a variable of the data type `class`.

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

//class:
class Fruit{
    //class properties
public:
    string name;
    string color;
};

int main()
{
    //defining the object of class fruit:
    //Method-1:  Object created on the stack
    Fruit apple; //object:
    apple.name = "Apple";
    apple.color = "Red";
    cout<<apple.name<<":"<<apple.color<<endl;
    //OUTPUT : Apple:Red

    //defining the object of class fruit:
    //Method-2: Object created on the heap
    Fruit *mango = new Fruit();
    mango->name = "Mango";
    mango->color = "Yellow";
    cout<<mango->name<<":"<<mango->color<<endl;
    //OUTPUT : Mango:Yellow

    return 0;
}
```



github/RIOLOG

Some Key Point to Notice realted to creation of object:-

1. Objects created with `new` have dynamic storage duration and persist until explicitly deallocated using `delete`.
2. Normal creation allocates memory on the stack, while `new` allocates memory on the heap.
3. Objects created normally have automatic storage duration (scoped within their block), while objects created with `new` have dynamic storage duration and persist until explicitly deallocated.
4. Using `new` allows more manual control over object lifetime but requires responsibility for managing memory allocation and deallocation.

Constructor:

- Constructors are used to initialize an object.
- Constructors are the special member functions that get invoked or called automatically when an object of the class is created.
- The constructor has the same name as the class name and does not have a return type.

Type of Constructor:

1 . Default Constructor: A default constructor doesn't take any arguments and is invoked when an object is created without any initialization parameters.

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

class Rectangle{
public:
    int length;
    int breadth;

    //default constructor:
    //no arg. passed:
    Rectangle()    //constructor same as class name:
    {
        length = 5;
        breadth = 5;
    }
};

int main()
{
    Rectangle R1;
    cout<<R1.length<<"-"<<R1.breadth<<endl;
    return 0;
}
//OUTPUT: 5-5
```



github/RIOLOG

2. Parameterized Constructor: A parameterized constructor takes parameters to initialize an object with specific values.

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

class Rectangle{
public:
    int length;
    int breadth;

    //Parameterized Constructor
    //arg. are passed:
    Rectangle(int x , int y)
    {
        length = x;
        breadth = y;
    }
};

int main()
{
    Rectangle R2(10,15);
    cout<<R2.length<<"-"<<R2.breadth<<endl;
    return 0;
}
//OUTPUT: 10-15
```



github/RIOLOG

3. Copy Constructor: A copy constructor creates a new object as a copy of an existing object of the same class.

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

class Rectangle{
public:
    int length;
    int breadth;

    //Parameterized Constructor
    //arg. are passed:
    Rectangle(int x , int y)
    {
        length = x;
        breadth = y;
    }

    //Copy Constructor:
    //Initialise an object by another existing object:
    Rectangle(Rectangle& R)
    {
        length = R.length;
        breadth = R.breadth;
    }
};

int main()
{
    Rectangle R2(10,15);
    //cout<<R2.length<<"-"<<R2.breadth<<endl;

    //Copy Constructor:
    Rectangle R3 = R2;
    cout<<R3.length<<"-"<<R3.breadth<<endl;

}
//OUTPUT: 10-15
```

```
#NOTE: A class can have multiple constructos.
```

Destructor:

- A destructor is a special member function within a class that is automatically called when an object goes out of scope or is explicitly deleted.
- Destructors have the same name as the class name in this fashion (~class_name).
- Destructors have no return type and no input parameters.

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

class Rectangle{
public:
    int length;
    int breadth;

    //Parameterized Constructor
    //arg. are passed:
    Rectangle(int x , int y)
    {
        length = x;
        breadth = y;
    }

    //Destructor:
    ~Rectangle()
    {
        cout<<"Destructor is called"<<endl;
    }

};

int main()
{
    Rectangle R2(10,15);
    cout<<R2.length<<"-"<<R2.breadth<<endl;
}
//OUTPUT:
//10-15
//Destructor is called
```

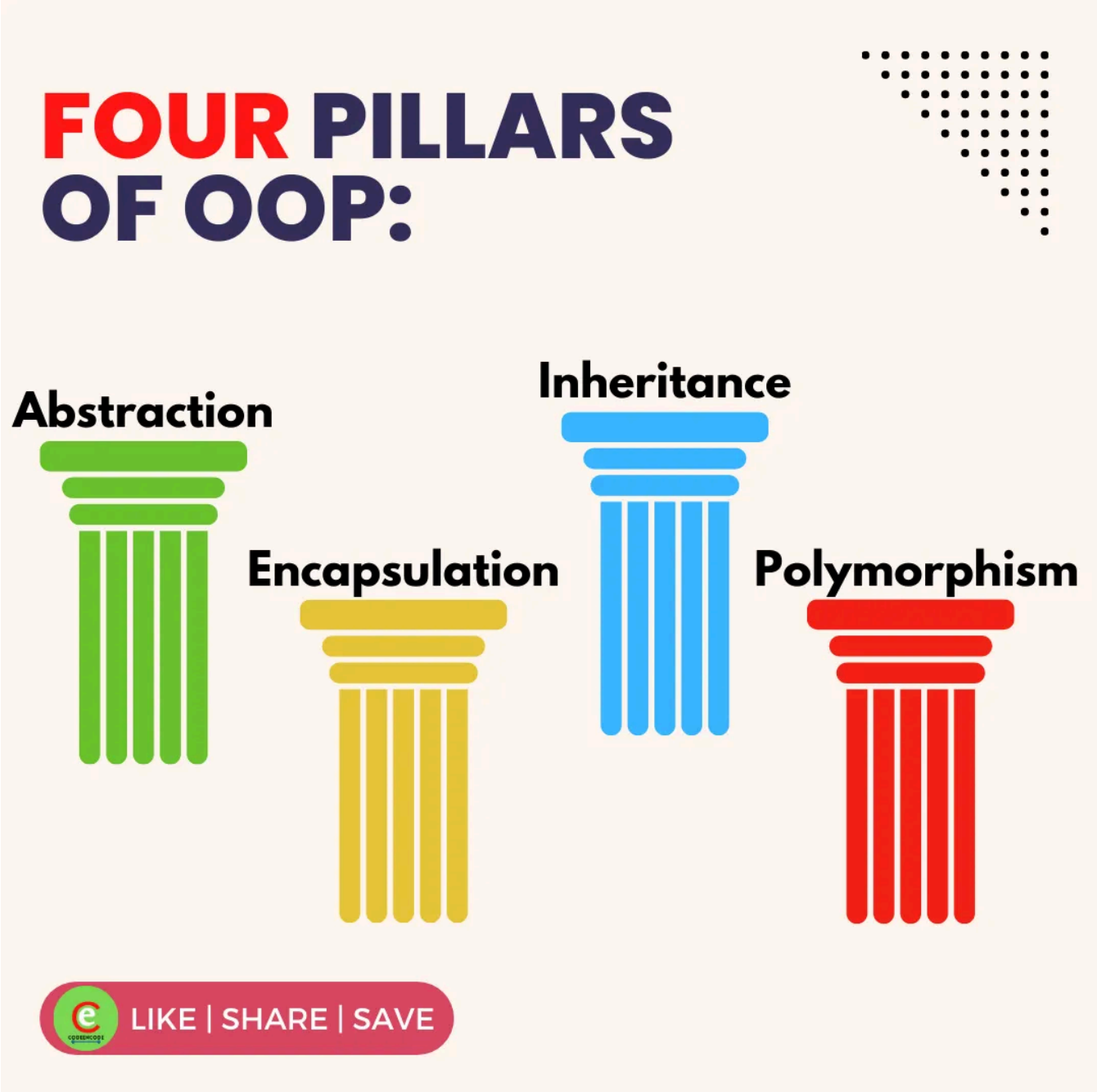


github/BIOLOG

Purpose of Destructors:

1. **Resource Deallocation:** Release resources acquired during the object's lifetime (like memory, file handles, database connections).
2. **Cleanup Operations:** Perform any necessary cleanup tasks before the object is destroyed.
3. **Implicit vs. Explicit** Invocation: Destructors are automatically called when an object goes out of scope. If an object is created with `new` its destructor is called explicitly with `delete.`

The four pillars of OOPs (object-oriented programming):



1. Encapsulation:

- Encapsulation in C++ is a fundamental concept in Object-Oriented Programming (OOP) that involves bundling the data (attributes/variables) and the methods (functions) into a single unit(class).
- The internal workings of an object are hidden from the outside world, providing controlled access to the object's members.
- Encapsulation also leads to data abstraction or data hiding, because of which the class is known as Abstract data type (ADT).

Example Without Encapsulation:



Example With Encapsulation:

OOP in C++

```cpp
#include<iostream>
using namespace std;

class Car {
private:
    string brand;
    int year;

public:
    // Setter methods
    void setBrand(const std::string& carBrand)
    {
        brand = carBrand;
    }

    void setYear(int carYear)
    {
        year = carYear;
    }

    // Getter methods
    string getBrand() const
    {
        return brand;
    }

    int getYear() const {
        return year;
    }
};

int main() {
    Car myCar;

    // Using setter methods to set data
    myCar.setBrand("Toyota");
    myCar.setYear(2020);

    // Using getter methods to access data
    cout << "Brand: " << myCar.getBrand() << ", Year: " << myCar.getYear() << std::endl;

    return 0;
}

//OUTPUT: Brand: Toyota, Year: 2020
```



github/RIOLOG

Points to Note:

- **Without Encapsulation:** In the first example, data members (`brand` and `year`) are public, allowing direct access and modification from outside the class, which can lead to uncontrolled changes and potential issues (e.g., inconsistent state).

- **With Encapsulation:** In the second example, data members (`brand` and `year`) are declared private, and inaccessible directly from outside the class. Access is provided through setter methods (`setBrand`, `setYear`) and getter methods (`getBrand`, `getYear`). This way, the internal data is encapsulated and can only be accessed or modified through controlled interfaces.

`Getter and Setter Functions:`

- **Getter Function:** this is a special function that allows us to access(get) the data members of a class(private and protected members).

- **Setter Function:** this is a special function that allows us to set the values to data members of an object of a class(private and protected members).

Advantages of Encapsulation:

1. **Controlled Access (Security):** Prevents direct modification of internal data, ensuring consistency and security.

2. **Flexibility and Maintainability:** Allows easy modification of internal implementation without affecting external code (e.g., if internal data representation changes).

3. **Reusability:** Encapsulated classes can be easily reused in different parts of a program or different programs, promoting code reusability.

2. Abstraction:

- Abstraction in C++ and Object-Oriented Programming (OOP) refers to the concept of hiding complex implementation details while providing a simplified interface to the users.

- It involves focusing on essential aspects and ignoring unnecessary details.

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

class Circle {
private:
    double radius;

public:
    // Setter method for radius
    void setRadius(double r
    ) {
        radius = r;
    }

    // Getter method for area (abstracting the calculation)
    double getArea() const
    {
        return 3.14159 * radius * radius;
    }
};

int main()
{
    Circle myCircle;
    myCircle.setRadius(5.0);

    // Area calculation using abstraction (getArea method)
    double area = myCircle.getArea();
    cout << "Area: " << area << std::endl;

    return 0;
}
//OUTPUT: Area: 78.5397
```



github/RIOLOG

Points to Note:

Get Ankit Singh's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

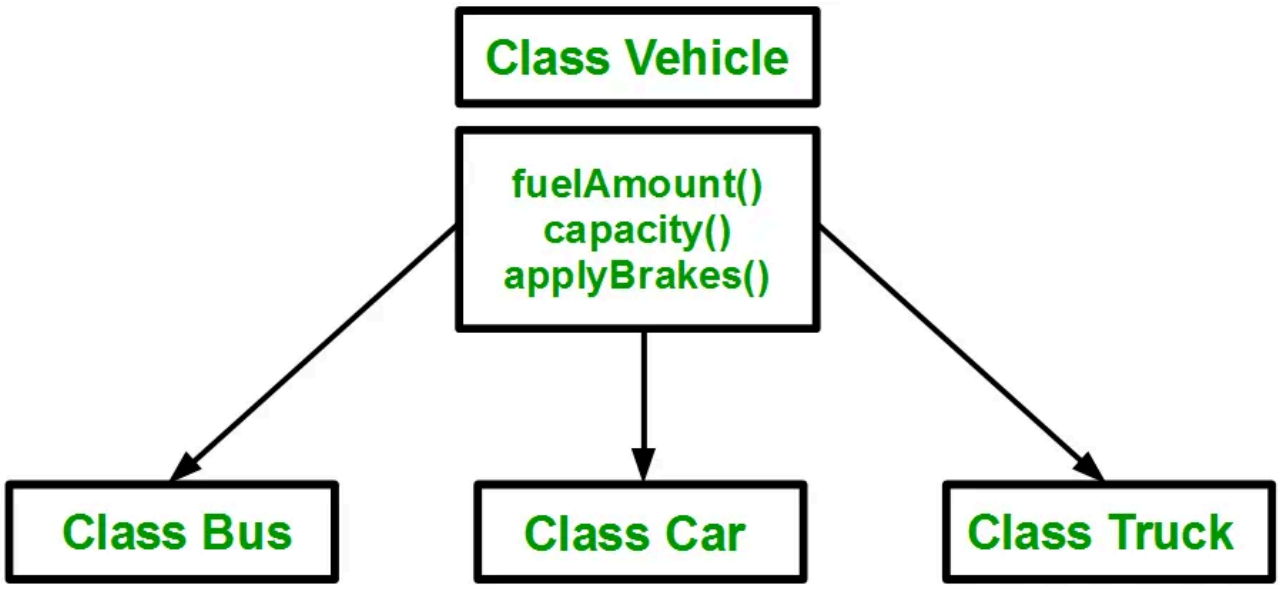
Remember me for faster sign in

With Abstraction: In the example, the `Circle` class encapsulates the `radius` as a private member and provides methods (`setRadius` and `getArea`) to manipulate and access the radius and compute the area. The area calculation logic is abstracted inside the `getArea` method, hiding the implementation details from the user.

Advantages of Abstraction:

1. **Simplification:** Users interact with simplified interfaces without worrying about underlying complexities.
2. **Modifiability:** Changes in the internal implementation do not affect users as long as the interface remains consistent.
3. **Security:** Encapsulation and abstraction together provide a secure way to access and manipulate data.

3. Inheritance:



- Inheritance in C++ and Object-Oriented Programming (OOP) is a mechanism that allows a class (derived or child class) to inherit properties and behavior from another class (base or parent class).
- Inheritance is the sharing of attributes and operations among classes based on a hierarchical relationship.
- In C++, inheritance means one object gets all the qualities and actions of its parent object. This helps you use, add to, or change the features and actions defined in another class.

> *Before diving into inheritance, let's talk about some related topics that affect how inheritance works between classes.*

Access Specifiers and Modes of Inheritance:

- In C++ OOP, access specifiers (public, private, and protected) control the visibility of class members (attributes and methods) in inheritance hierarchies.
- They determine how these features get passed on to other classes that inherit them.

1. **Public Access Specifier:**

- If data and methods of any class are public then they can be accessed from anywhere in a code.
- Accessible by anyone, including derived classes.

2. Protected Access Specifier:

- If data and methods of any class are protected, then they can be accessible in their class, parent class, and derived class.

3. Private Access Specifier:

- If data and methods of any class are private, then they are not accessible by derived classes.

```
NOTE: By Default access specifier of class is private.
```

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

class Parent{
public:
    int x;
protected:
    int y;
private:
    int z;
};

//child1 class is inheriting the parent
//class publicly.
class Child1: public Parent{
    //x will remain public     (can be accessible)
    //y will remain protected  (can be accessible)
    //z will remain private    (not be accessible)
}

//child2 class is inheriting the parent
//class privately.
class Child2: private Parent{
    //x will be private     (can be accessible)
    //y will be private     (can be accessible)
    //z will be private     `(not be accessible)
}

///child3 class is inheriting the parent
//class protectively.
class Child3: protected Parent{
    //x will be protected     (can be accessible)
    //y will be protected     (can be accessible)
    //z will be private       (not be accessible)
}

int main()
{
    Parent p;
    p.x;    //public then they can be accessed from anywhere in a code.
}
```



github/RIOLOG

NOTE: In child class 2 when we were inheriting the parent class privately all the variables(x,y,z) will remain inaccessible from outside of class.

- **`public` Inheritance:** `PublicDerived` class publicly inherits from `Base`. Public and protected members of `Base` remain public and protected, respectively, in `PublicDerived`.
- **`private` Inheritance:** `PrivateDerived` class privately inherits from `Base`. All members of `Base` become private in `PrivateDerived`.
- **`protected` Inheritance:** `ProtectedDerived` class protects the inheritance. All members of `Base` become protected in `ProtectedDerived`.

> *Here is the overview of this:*

Base class member access specifier	Type of Inheritence		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible (Hidden)	Not accessible (Hidden)	Not accessible (Hidden)

TYPE OF INHERITANCE:

Inheritance in C++

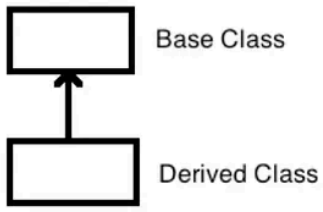


Fig. Single level inheritance

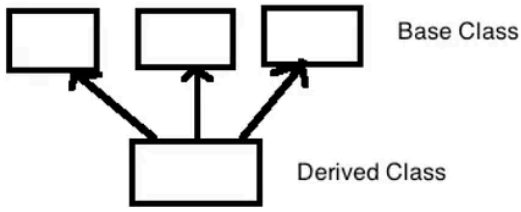


Fig. Multiple Inheritance

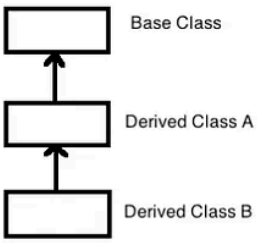


Fig. Multi level inheritance

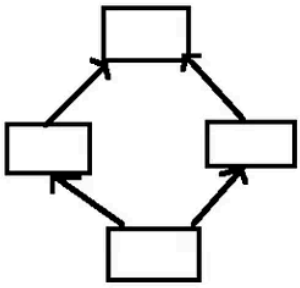


Fig. Hybrid Inheritance

1. **Single Inheritance:**

- In single inheritance, a derived class inherits from only one base class.

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

class Parent{
public:
    Parent()
    {
        cout<<"PARENT CLASS"<<endl;
    }
};

class Child: public Parent{
public:
    Child()
    {
        cout<<"CHILD CLASS"<<endl;
    }
};

int main()
{
    Child c;
    return 0;
}

//OUTPUT:
// PARENT CLASS
// CHILD CLASS
```



github/RIOLOG

2. Multilevel Inheritance:

- In multilevel inheritance, a derived class becomes the base class for another derived class, forming a chain.

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

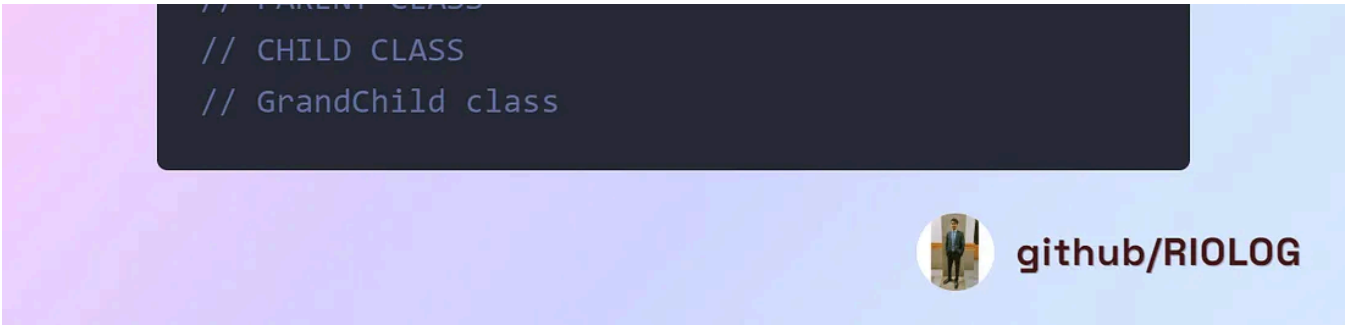
class Parent{
public:
    Parent()
    {
        cout<<"PARENT CLASS"<<endl;
    }
};

class Child: public Parent{
public:
    Child()
    {
        cout<<"CHILD CLASS"<<endl;
    }
};

class GrandChild: public Child{
public:
    GrandChild()
    {
        cout<<"GrandChild class"<<endl;
    }
};

int main()
{
    GrandChild gc;
    return 0;
}

//OUTPUT:
// PARENT CLASS
```



3. Multiple Inheritance:

- Multiple inheritance involves a derived class inheriting from more than one base class.

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

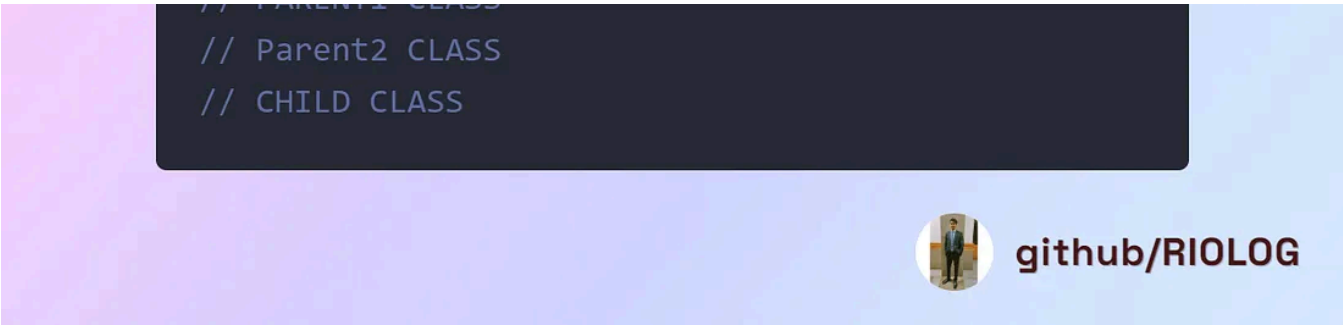
class Parent1{
public:
    Parent1()
    {
        cout<<"PARENT1 CLASS"<<endl;
    }
};

class Parent2{
public:
    Parent2()
    {
        cout<<"Parent2 CLASS"<<endl;
    }
};

class Child: public Parent1, public Parent2{
public:
    Child()
    {
        cout<<"CHILD CLASS"<<endl;
    }
};

int main()
{
    Child c;
    return 0;
}

//OUTPUT:
// PARENT1 CLASS
```



4. Hierarchical Inheritance:

- Hierarchical inheritance involves multiple derived classes inheriting from a single base class.

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

class Parent1{
public:
    Parent1()
    {
        cout<<"PARENT1 CLASS"<<endl;
    }
};

class Child1: public Parent1{
public:
    Child1()
    {
        cout<<"Child1 CLASS"<<endl;
    }
};

class Child2: public Parent1{
public:
    Child2()
    {
        cout<<"Child2 CLASS"<<endl;
    }
};

int main()
{
    Child1 c1;
    Child2 c2;
    return 0;
}

//OUTPUT:
```



5. Hybrid Inheritance:

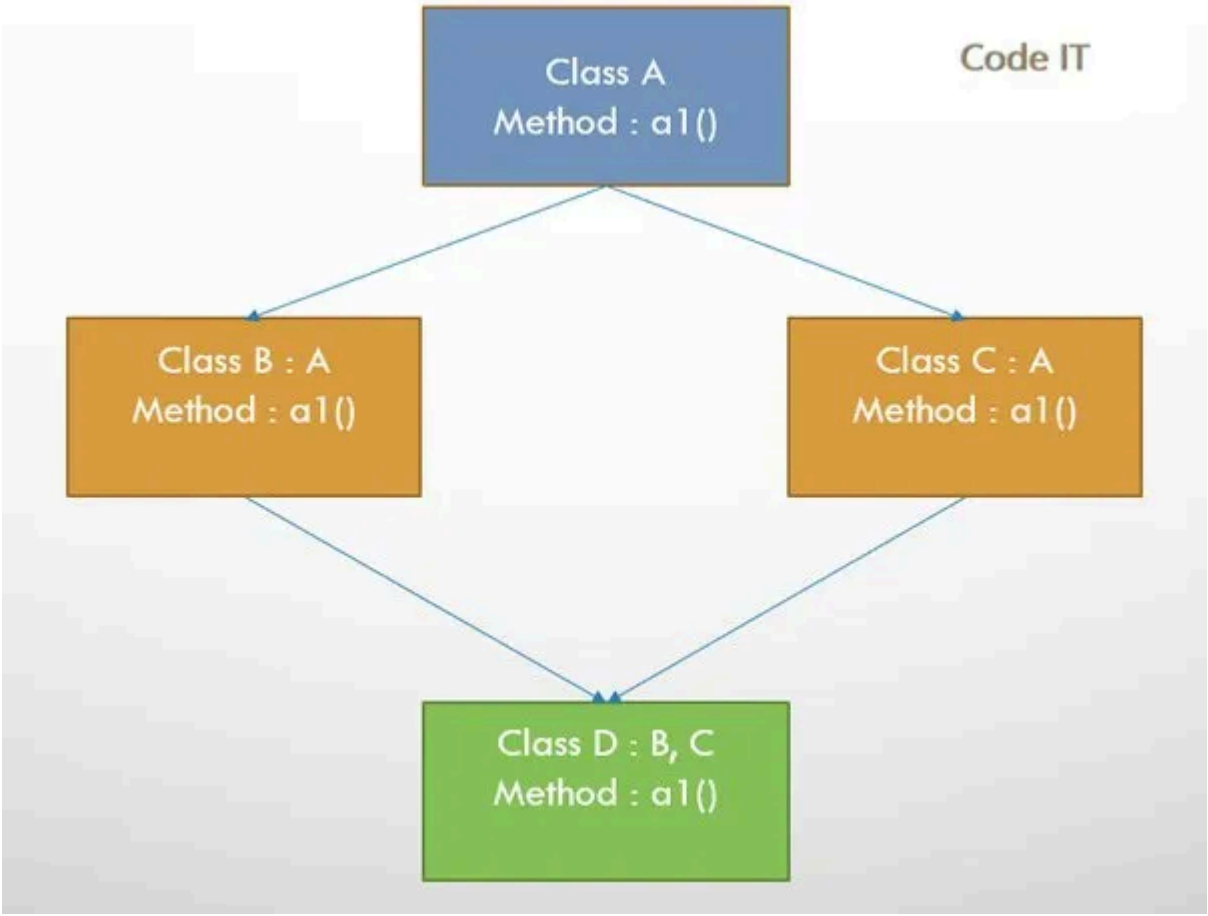
- Hybrid inheritance is a combination of multiple inheritance and other forms of inheritance.

> *There's a well-known issue in inheritance called the diamond problem. Let's grasp what it's about.*

Diamond Problem Scenario: (Inheritance Ambiguity):

The "**diamond problem**" happens in multiple inheritance when a class comes from two classes with a shared ancestor.

It causes confusion in accessing inherited parts and makes the inheritance setup more complicated, leading to potential problems.


Code IT
Class A
Method : a1()
Class B : A
Method : a1()
Class C : A
Method : a1()
Class D : B, C
Method : a1()

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

class Parent{
public:
    Parent()
    {
        cout<<"PARENT CLASS"<<endl;
    }
};

class Child1: public Parent{
public:
    Child1()
    {
        cout<<"Child1 CLASS"<<endl;
    }
};

class Child2: public Parent{
public:
    Child2()
    {
        cout<<"Child2 CLASS"<<endl;
    }
};

class GrandChild: public Child1, public Child2{
public:
    GrandChild()
    {
        cout<<"GrandChild class"<<endl;
    }
};
```

```
int main()
{
    GrandChild gc;
    return 0;
}

//OUTPUT:
// PARENT CLASS
// Child1 CLASS
// PARENT CLASS
// Child2 CLASS
// GrandChild class
```



github/RIOLOG

Important things to remember are:

- In the above example the parent class constructor is called Twice.
- One time the parent class constructor is called by child 1 class and next time it is called by child 2 class.

> `NOTE:` *To resolve the diamond problem, you can use virtual inheritance. By using the* `virtual` *keyword when inheriting from the common base class, it ensures that only one instance of the base class is shared among the derived classes.*

4. Polymorphism:

- ability of objects/methods to take different forms.
- Polymorphism in C++ is a fundamental concept in Object-Oriented Programming (OOP) that allows objects of different classes to be treated as objects of a common superclass
- Polymorphism is achieved through function overriding and function overloading.

Example: “Right” word can be used in “Right/Wrong” scenario as well as in “Left/Right” scenario.

Types of Polymorphism:

1. Compile-Time Polymorphism (Static Binding):

a. Function Overloading:

- Function overloading allows multiple functions with the same name but different parameters or parameter types within the same scope.

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

class Sum{
public:
    void add(int x, int y)   //METHOD-1:
    {
        int sum = x + y;
        cout<<sum<<endl;
    }

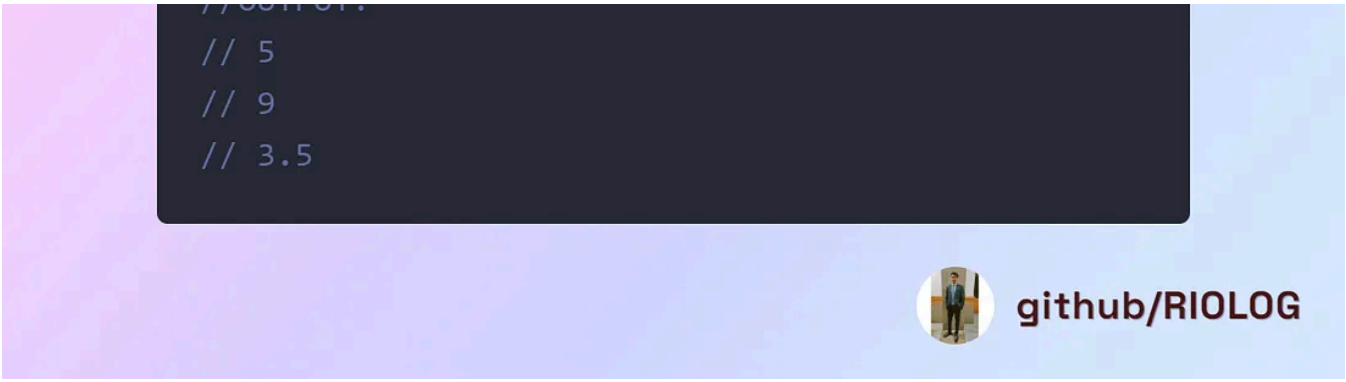
    void add(int x, int y, int z)   //METHOD-2:
    {
        int sum = x + y + z;
        cout<<sum<<endl;
    }

    void add(float x, float y)   //METHOD-3:
    {
        float sum = x + y;
        cout<<sum<<endl;
    }
};

int main()
{
    Sum s;
    s.add(2,3);         //METHOD-1:
    s.add(2,3,4);       //METHOD-2:
    s.add(float(1.2), float(2.3));   //METHOD-3:

    return 0;
}

//OUTPUT:
```



b. Operator Overloading:

- Operator overloading enables operators like `+`, `-`, `*`, etc., to be overloaded for user-defined types.

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

class Complex{
public:
    int real;
    int img;

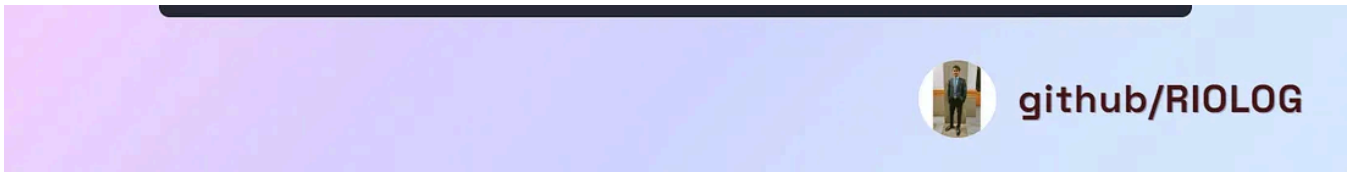
    Complex(int x, int y)
    {
        real = x;
        img = y;
    }
    Complex operator+ (Complex &c)
    {
        Complex ans(0,0);
        ans.real = real + c.real;
        ans.img = img + c.img;
        return ans;
    }
};

int main()
{
    Complex c1(1,2);
    Complex c2(1,3);

    Complex c3 = c1 + c2;
    cout<<c3.real<<" i"<<c3.img<<endl;

    return 0;
}

//OUTPUT: 2 i5
```



2. Run-Time Polymorphism (Dynamic Binding):

a. Function Overriding (Virtual Functions):

- Function overriding allows a derived class to provide a specific implementation of a function that is already defined in its base class. It's achieved using virtual functions.
- child class defines the same method(function), that is defined in `paremt` class.

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

class Parent{
public:
    virtual void Print()
    {
        cout<<"PATENT CLASS"<<endl;
    }
    void show()
    {
        cout<<"Parent show"<<endl;
    }
};

class Child: public Parent
{
    void Print()
    {
        cout<<"Child CLASS"<<endl;
    }
    void show()
    {
        cout<<"Child show"<<endl;
    }

};

int main()
{
    Parent *p;
    Child c;

    p = &c;
    p->Print();
```

```
    p->show();

    return 0;
}

//OUTPUT:
// Child CLASS
// Parent show
```



github/RIOLOG

Advantages of Polymorphism:

- **Flexibility and Extensibility:** Simplifies adding new classes without modifying existing code.
- **Code Reusability:** Allows for writing generic code that can work with multiple derived types.
- **Enhanced Maintainability:** Supports clearer and more concise code structure.

Friend Function:



- A Frined Function is a non-member function which can access the private member of a class.
- It's often used to provide specific functionalities that require access to private members but are not part of the class itself.

Example:

OOP in C++

```cpp
#include<iostream>
using namespace std;

class A{
    int x ;
public:
    A (int y)
    {
        x = y;
    }
    friend void print(A &obj);
};

//Friend Function:
void print(A &obj){
    cout<<obj.x<<endl;
}


int main()
{
    A obj(10);
    print(obj);
    return 0;
}

//OUTPUT: 10
```



github/RIOLOG

Conclusion:

In C++, mastering Object-Oriented Programming opens doors to creating strong, scalable, and manageable code. Understanding encapsulation, inheritance, polymorphism, and abstraction is crucial for effective software design.

Learning about classes, objects, access modifiers, and advanced topics such as templates, exception handling, and design patterns marks the beginning of becoming skilled in C++ programming.

With this guide as a strong base, you're ready to handle the complexities of Object-Oriented Programming in C++ and solve real-world problems creatively.

Happy coding!



> *"Level Up Your Code: C++ OOP Masterclass!"*

Oop Concepts + Cpp + Software Engineering + Programming +